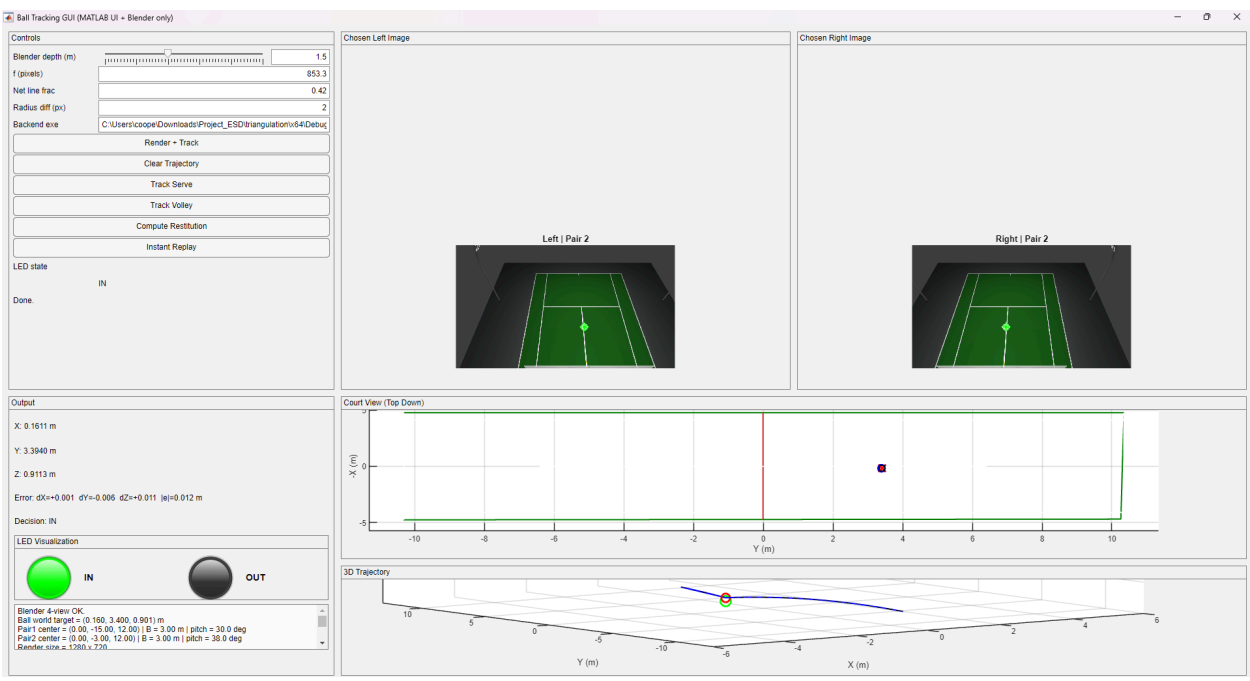
Tennis Ball Tracking System

User Manual

Version 1.1
April 2026
ESD2 Capstone Project



Preface

This User Manual presents the major features of the Tennis Ball Hawk-Eye System, including the MATLAB graphical user interface, Blender rendering environment, and C++/OpenCV triangulation backend. This manual may be used by anyone with access to the project files.

The purpose of this manual is to provide a complete guide to understanding, configuring, operating, and troubleshooting the system. It explains how the system components interact, how to start each required program, how to run the GUI features, how to adjust important parameters, and how to interpret the output data.

Revision History

Date	*Revision*	*Description*	*Author*
04/26/2026	1.0	Initial user manual created for the MATLAB GUI, Blender rendering environment, and C++/OpenCV backend	C. Felle
04/26/2026	1.1	Added to manual for User Facing website using Python and MySQL.	J. Charnock

Table of Contents

1. Introduction
  1.1 Related Documents
  1.2 Functional Description

2. Orientation
  2.1 Standard Conventions
  2.2 Project Folder Structure
  2.3 System Workflow
  2.4 GUI Button Structure
    2.4.1 Render + Track
    2.4.2 Track Serve
    2.4.3 Track Volley
    2.4.4 Instant Replay
    2.4.5 Compute Restitution
    2.4.6 Run Validation
    2.4.7 Run Camera Accuracy
    2.4.8 Clear

3. System Setup
  3.1 Required Software
  3.2 MATLAB Setup
  3.3 Blender Setup
  3.4 Backend Setup
  3.5 Folder and Path Parameters

4. Initial Use
  4.1 Starting the Full System

5. GUI Configuration
  5.1 Professional GUI
  5.2 Development GUI
  5.3 Adjustable GUI Parameters

6. Blender Rendering Environment

6.1 Blender Scene Requirements
6.2 Camera Setup

7. Backend and Triangulation
7.1 Backend Input Files
7.2 Camera and Intrinsic Parameters
7.3 Detection Parameters
7.4 Stereo Triangulation Parameters
7.5 Tracking Range Parameters

8. Shot Classification
8.1 Court Geometry
8.2 Bounce Detection
8.3 IN / OUT / NET Decision

9. Website and SQL Output
9.1 lastShot Structure
9.2 SQL Structure

10. User Sided GUI
10.1 Basic Requirements
10.2 MySQL Database
10.3 Client Facing Site

11. Troubleshooting

Glossary

Appendices
A.1 Recommended Demo Procedure
A.2 Important Parameters Checklist

1. Introduction

The Tennis Ball Tracking System is a software-based simulation and tracking platform designed to classify tennis shots as IN, OUT, or NET using stereo camera geometry. The system uses a MATLAB graphical user interface as the main operator control panel, a Blender environment to simulate the tennis court and camera views, and a C++/OpenCV backend to detect and triangulate the tennis ball position.

The system is intended to demonstrate how stereo vision can be used to estimate a tennis ball's three-dimensional location and determine whether a shot landed inside or outside the court boundary. The MATLAB GUI controls the overall workflow, Blender produces synchronized camera images, and the backend processes those images to estimate the ball location.

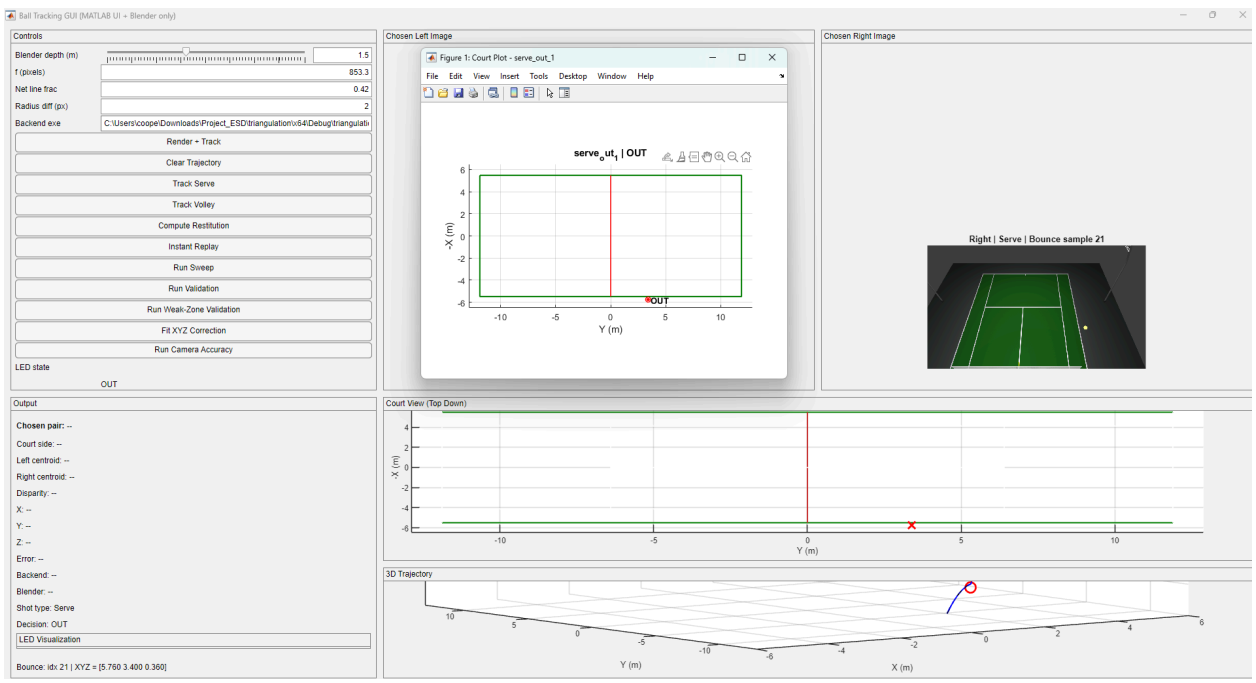


Fig. 1.1: Final system output showing GUI, rendered camera views, and court decision plot

1.1 Related Documents

The following project files and documents may be used with this manual:

GUI_Professional.m
Project_GUI_Development_v1.m
blenderLink.m

blenderServer.py
Triangulation.exe
sql_link.m
tennis_GreenCourt
tennis_RedCourt
tennis_BlueCourt
README.txt
Capstone CDR Documentation
Project Validation Results
Website / SQL Integration Notes

The MATLAB GUI files are the primary user-facing scripts. The Blender server file handles communication between MATLAB and Blender. The C++ backend executable performs detection and triangulation.

1.2 Functional Description

The Tennis Ball Tracking System consists of several interrelated modules:

1. MATLAB GUI Module
2. Blender Rendering Module
3. MATLAB-to-Blender Communication Module
4. C++/OpenCV Backend Module
5. Stereo Triangulation Module
6. Shot Classification Module
7. Validation and Accuracy Testing Module
8. Website / SQL Output Module

The MATLAB GUI Module allows the user to run tests using buttons such as **Render + Track**, **Track Serve**, **Track Volley**, **Instant Replay**, **Run Validation**, and **Compute Restitution**.

The Blender rendering module creates the simulated tennis environment, including the court, net, tennis ball, lighting, and cameras.

The MATLAB-to-Blender communication module sends camera and ball position commands from MATLAB to Blender and receives said rendered image frames.

The C++/OpenCV backend module detects the ball in stereo images and calculates its measured position.

The stereo triangulation module uses camera parameters, focal length, baseline, principal point, and disparity to estimate X, Y, and Z coordinates.

The shot classification module determines whether a shot is **IN**, **OUT**, or **NET** based on the bounce location and court geometry.

The website / SQL output module prepares selected shot information for database or web display.

2. Orientation

This section explains the basic structure of the system and the conventions used throughout the manual.

For this project, the orientation section explains how the user moves through the GUI, what each button does, where output files are saved, and which parameters must be checked before running the system.

2.1 Standard Conventions

The following conventions are used in this manual:

MATLAB command:
A command that is typed into the MATLAB Command Window.

File path:
A folder or file location that *may* (likely will) need to be changed for each computer.

Parameter:
A value inside the MATLAB GUI, backend config file, or Blender scene that affects the tracking of the ball.

Output:
A generated file, plot, image, text result, or structure.

Example MATLAB commands are shown in code format:

GUI_Professional

Example folder paths are shown in text format:

C:\Users\coope\Downloads\Project_ESD

Values that may require adjusting are identified as adjustable parameters.

2.2 Project Folder Structure

The project should be organized so that the GUI, Blender files, backend files, and output folders can be located easily. See example below:

```
Project_ESD/
│
├── GUI_Professional.m
├── Project_GUI_Development_v1.m
├── blenderLink.m
│
├── blender/
│   └── blenderServer.py
│
├── triangulation/
│   └── x64/
│       └── Debug/
│           └── triangulation.exe
│
├── tracker_runs/
├── webdev/
│   └── flask_tracker/
│       └── backend/
│       └── frontend/
│       └── install.sh
│       └── startup.sh
│
├── court_outputs/
├── website_upload/
├── demo_case_library/
└── config.txt files
```

Downloads > Project_ESD >

Sort View ...

Name	Date modified	Type	Size
Today			
pair1_left_mask	4/28/2026 9:09 AM	PNG File	3 KB
pair1_right_mask	4/28/2026 9:09 AM	PNG File	3 KB
pair2_left_mask	4/28/2026 9:09 AM	PNG File	3 KB
pair2_right_mask	4/28/2026 9:09 AM	PNG File	3 KB
tracker_runs	4/28/2026 9:09 AM	File folder	
Earlier this week			
README	4/26/2026 2:40 PM	Text Document	15 KB
sql_link	4/26/2026 7:26 PM	MATLAB Code	2 KB
GUI_Professional	4/26/2026 7:28 PM	MATLAB Code	91 KB
pushShotToWebsite	4/26/2026 7:26 PM	MATLAB Code	3 KB
buildWebPayload	4/26/2026 6:58 PM	MATLAB Code	9 KB
Project_GUI_Development_v1	4/26/2026 7:28 PM	MATLAB Code	91 KB
images	4/26/2026 2:52 PM	File folder	
Last week			
tennis_BlueCourt	4/20/2026 5:20 PM	BLEND File	8,125 KB
tennis_GreenCourt	4/20/2026 5:20 PM	BLEND File	8,125 KB
tennis_RedCourt	4/20/2026 5:20 PM	BLEND File	8,125 KB
uploadLastShotToWebsite	4/19/2026 4:08 PM	MATLAB Code	2 KB
runShotSweep	4/21/2026 3:40 PM	MATLAB Code	4 KB
getShotPreset	4/21/2026 3:40 PM	MATLAB Code	3 KB
triangulation	4/21/2026 3:22 PM	File folder	
Earlier this month			
rebuild_all_cases_manifest	4/15/2026 6:34 PM	MATLAB Code	5 KB
build_demo_case_library_fixed	4/15/2026 6:04 PM	MATLAB Code	7 KB
generate_case_image_sequence_fixed	4/15/2026 6:01 PM	MATLAB Code	7 KB
track_case_from_images	4/16/2026 2:57 PM	MATLAB Code	15 KB
track_case_set_from_manifest	4/16/2026 12:34 PM	MATLAB Code	3 KB
run_cor_surface_comparison	4/15/2026 5:12 PM	MATLAB Code	4 KB
run_cor_bounce_test	4/15/2026 5:03 PM	MATLAB Code	12 KB

Fig.2: Project_ESD folder opened in File Explorer

2.3 System Workflow

The full system workflow is:

The user normally begins by opening Blender and running the Blender server. Then the user opens MATLAB, runs the GUI, performs a single Render + Track test, and then runs a serve or volley tracking test.

2.4 GUI Button Structure

The main GUI options are:

Render + Track
Track Serve
Track Volley

Instant Replay
Compute Restitution
Run Validation
Run Camera Accuracy
Clear

2.4.1 Render + Track

The Render + Track option is used to run a single position tracking test. This is usually the first button that should be used after starting Blender, running the Blender server, and opening the MATLAB GUI. It verifies that the main system pipeline is working correctly.

When this button is selected, MATLAB places the ball at a predefined test location, sends the ball and camera information to Blender, receives the rendered stereo images, saves those images to the tracking run folder, and calls the C++/OpenCV backend. The backend detects the ball in the left and right images, triangulates the measured 3D position, and writes the result to a text file. MATLAB then reads the result and displays the measured position in the GUI.

This option is necessary for confirming that the Blender connection, backend executable path, camera parameters, and detection settings are correct before running a full serve or volley.

Important parameters:

-ballXYZ
-projectRoot
-backendExe
-runRoot
-configPath
-resultPath
-image_width
-image_height
-fx_px
-fy_px
-cx
-cy
-pair1_baseline_m
-pair2_baseline_m
-groundZ

Expected output:

-Rendered left/right camera images
-Backend result.txt file
-Measured X, Y, Z position
-Selected stereo pair
-Position error values, if true position is known
-Updated GUI image display

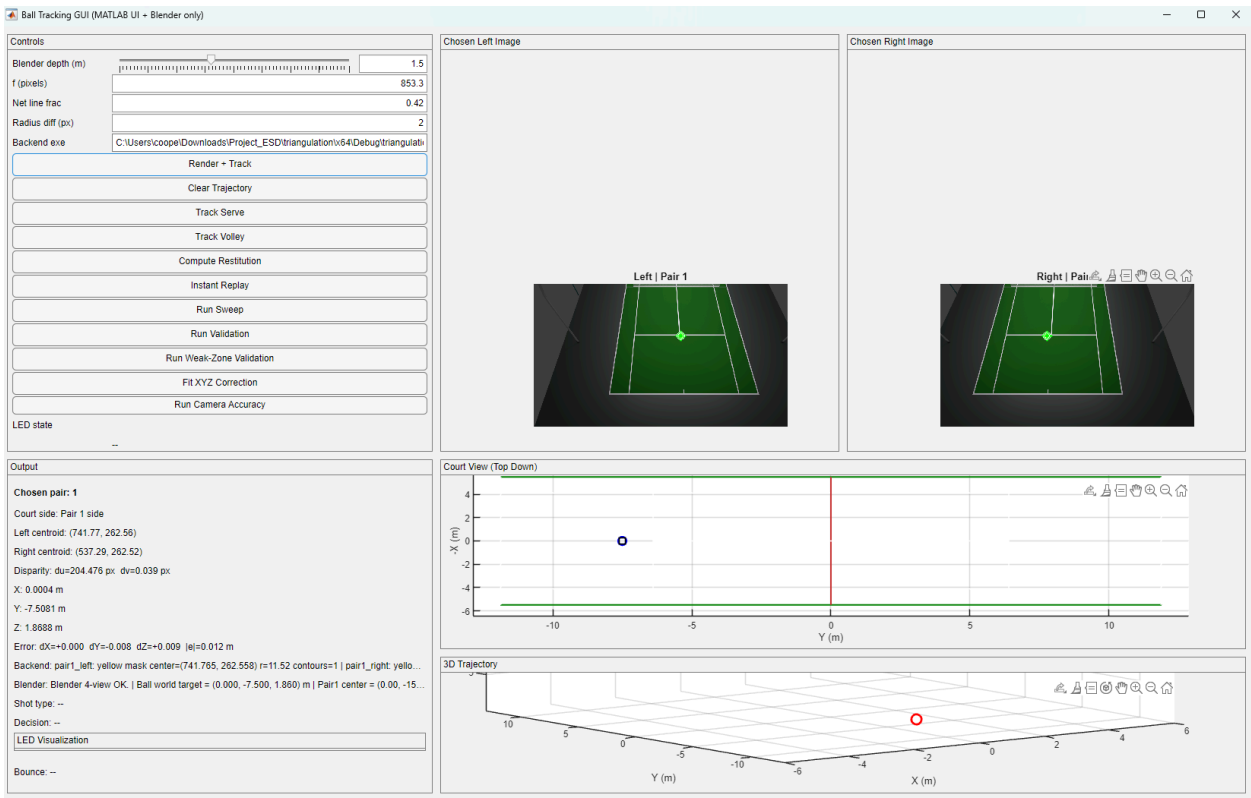


Fig.3: Render + Track button and resulting measured XYZ output

2.4.2 Track Serve

The Track Serve option generates and tracks a simulated tennis serve. This option is used to display the full shot-tracking workflow over multiple time steps instead of a single static ball position.

When the button is pressed, MATLAB creates a serve trajectory using predefined shot parameters. The ball position is updated over time, and Blender renders stereo images for each point along the trajectory. Each frame (or selected set of frames) is processed in the backend to estimate the ball's measured position. MATLAB then reconstructs the measured trajectory, identifies the bounce point, and classifies the serve as IN, OUT, or NET.

Important parameters:

-shotName
-shotNo
-shotType
-duration
-dt
-t
-xyzTrue
-xyzMeas
-groundZ
-courtGeom
-courtGeom.netY
-lineTolerance
-bounceIdx
-bounceXYZ
-selectedPair

Expected output:

-Serve trajectory
-Measured ball positions
-Bounce point
-IN / OUT / NET decision
-Selected stereo pair
-Saved court plot PNG
-Updated lastShot structure
-Optional website / SQL payload

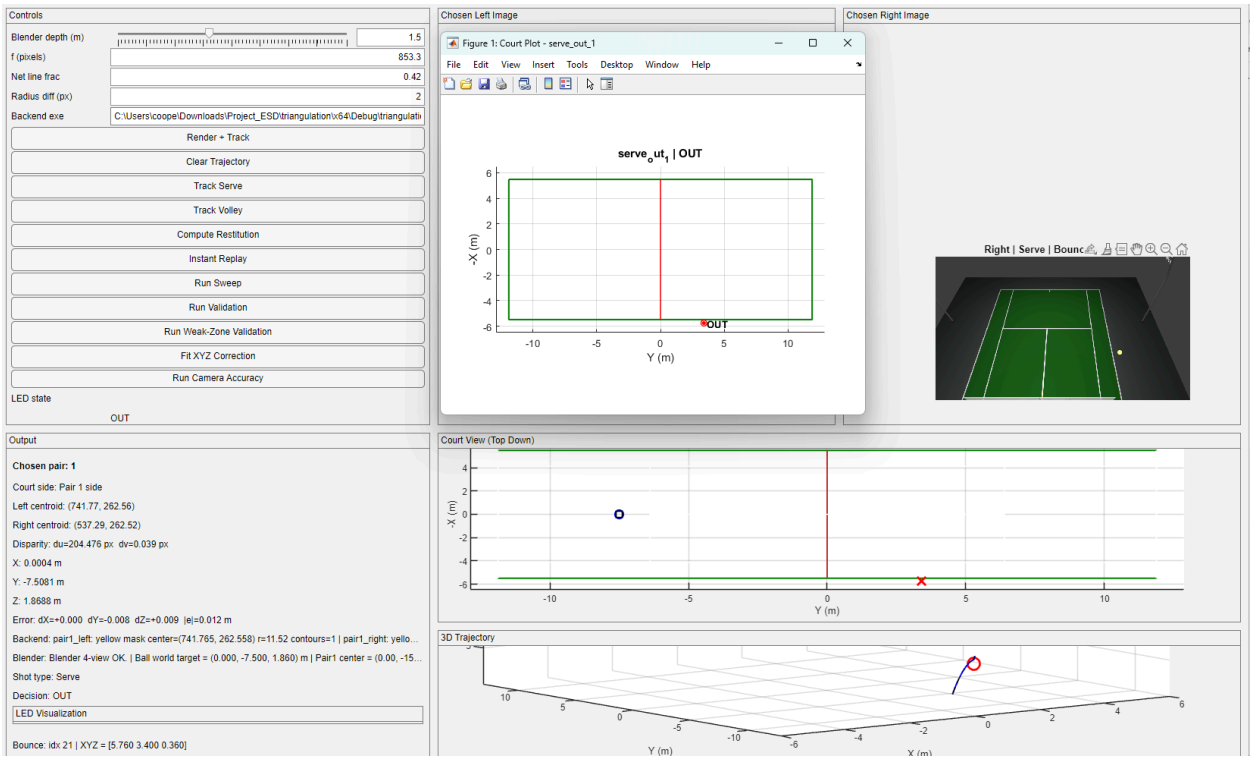


Fig. 4: Track Serve output with final court plot and decision label

2.4.3 Track Volley

Similar to the Track Serve function, Track Volley generates and tracks a simulated volley shot. It uses the same general pipeline as Track Serve, but the trajectory parameters are different because a volley starts and travels differently than a serve.

When this button is pressed, MATLAB generates a volley trajectory, sends the ball positions to Blender, receives the rendered stereo images, calls the backend, and reads the measured 3D positions. MATLAB then determines the bounce location and classifies the shot as IN, OUT, or NET.

This option is useful for showing that the system can classify more than one type of tennis shot. Since volleys typically have a different starting height, speed, and direction than serves, this test helps confirm that the tracking pipeline works across different ball trajectories.

Important parameters:

-shotName
-shotNo

-shotType
-duration
-dt
-initialPosition
-initialVelocity
-xyzTrue
-xyzMeas
-groundZ
-courtGeom
-lineTolerance
-bounceIdx
-bounceXYZ
-selectedPair

Expected output:

-Volley trajectory
-Measured ball positions
-Bounce point
-IN / OUT / NET decision
-Saved court plot PNG
-Updated lastShot structure

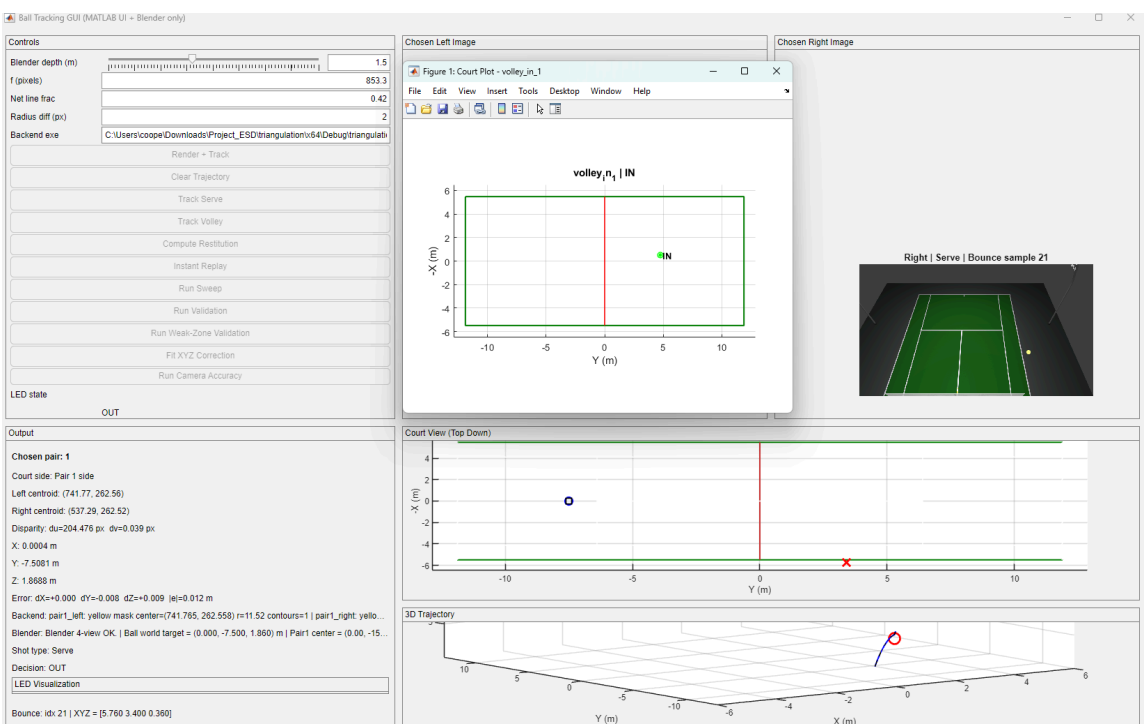


Fig. 5: Track Volley output with trajectory and bounce point

2.4.4 Instant Replay

The Instant Replay option replays the most recently tracked shot. This option depends on the lastShot structure, so a serve, volley, or other valid tracking operation **must** be completed before instant replay can be used.

When selected, MATLAB reads the stored trajectory data from lastShot and re-displays the motion of the ball through the GUI. This allows the user to review the shot without having to re-render all Blender frames or re-run the backend detection process.

Required stored data:

-lastShot.t
-lastShot.xyz
-lastShot.xyzMeas
-lastShot.xyzTrue
-lastShot.decision
-lastShot.bounceXYZ
-lastShot.bounceIdx
-lastShot.selectedPair
-lastShot.shotName
-lastShot.shotNo

Expected output:

-Replayed trajectory
-Previously calculated decision
-Previously identified bounce point
-No new backend run required

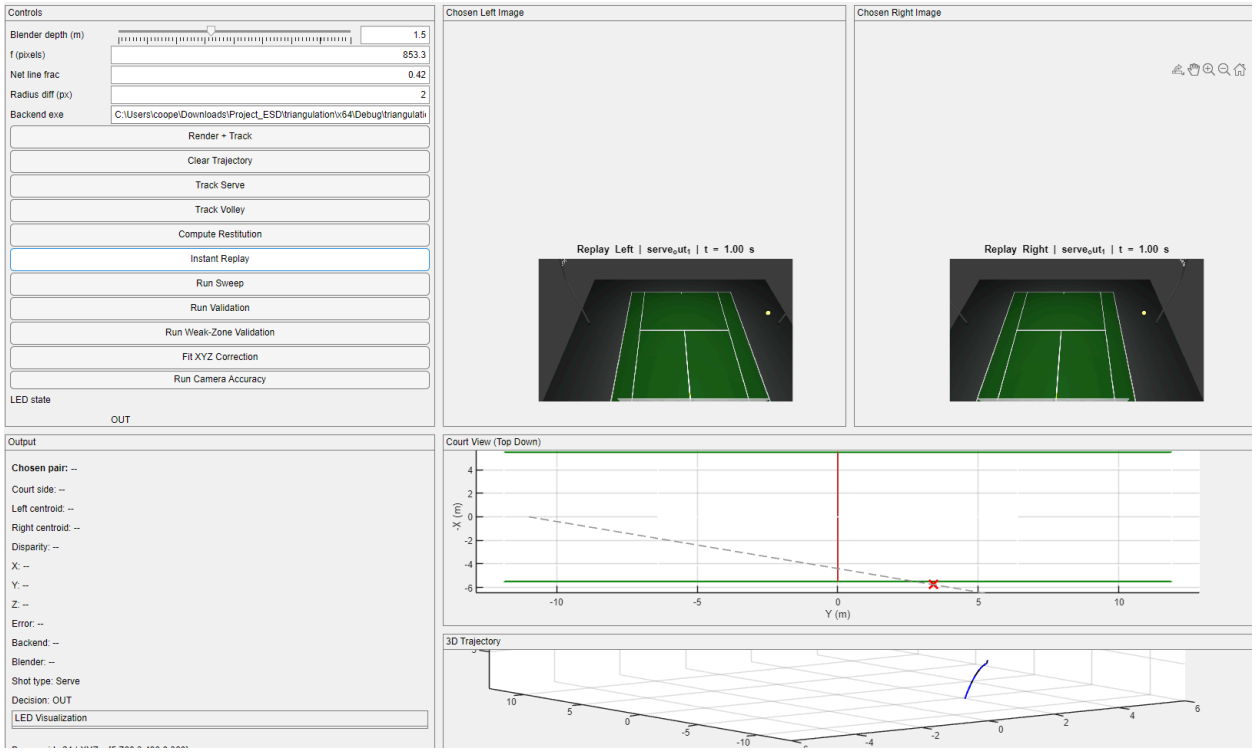


Fig. 6: Instant Replay display showing stored trajectory playback (for full video, refer to CDR presentation)

2.4.5 Compute Restitution

The Compute Restitution option calculates the coefficient of restitution (COR) from a bouncing ball trajectory. COR describes how much vertical velocity the ball retains after impact with the court surface (i.e. how bouncy a surface is).

When this option is selected, MATLAB analyzes the vertical motion of the ball before and after a bounce. The incoming vertical velocity and outgoing vertical velocity are estimated, and the coefficient of restitution is calculated as the ratio between them.

This option is useful for comparing simulated court behavior. A harder surface generally produces a higher bounce response, while a softer surface produces a lower bounce response. In the project, this feature helps connect the tracking system to court surface behavior and bounce analysis.

Important parameters:

-t
-xyz
-xyzMeas

-groundZ
-bounceIdx
-incomingVelocityZ
-outgoingVelocityZ
-COR

Expected output:

-Bounce time
-Incoming vertical velocity
-Outgoing vertical velocity
-Coefficient of restitution
-COR table or printed result
-Optional bounce plot

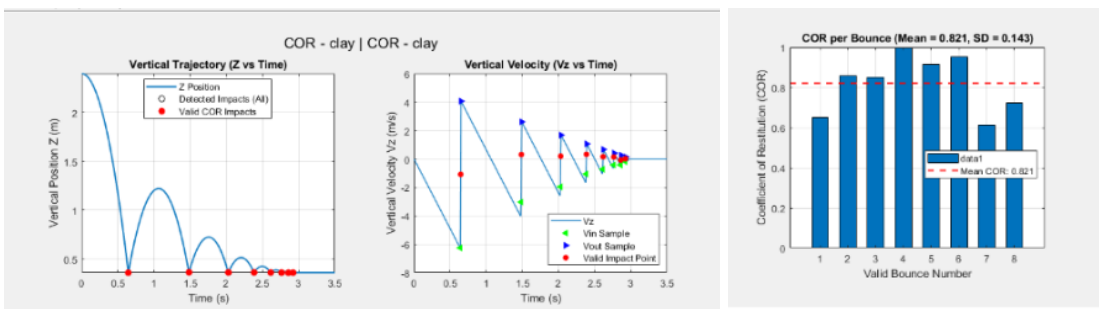


Fig. 7: Example COR output plots, clay court

2.4.6 Run Validation

The Run Validation option runs predefined test cases to evaluate system accuracy. This option is usually included in the development GUI rather than the final professional GUI.

When selected, MATLAB processes a group of known test cases and compares the system's predicted result against the expected result. These cases may include serves, volleys, static ball tests, different court colors, line-touching shots, out shots, and net shots.

Validation is used to measure both classification accuracy and position error. It helps determine whether the system is consistently identifying the correct result and whether the measured bounce point is close to the true bounce point.

Important parameters:

-demo_case_library
-manifest.csv
-expected_result
-predicted_result
-xyzTrue
-xyzMeas
-bounce_xy_true
-bounce_xy_measured
-meanError
-RMSE
-classificationAccuracy

Expected output:

-Validation table
-Pass/fail result for each test case
-Classification accuracy
-Position error statistics
-Failure map
-Error plots
-Court color comparison plots, if enabled

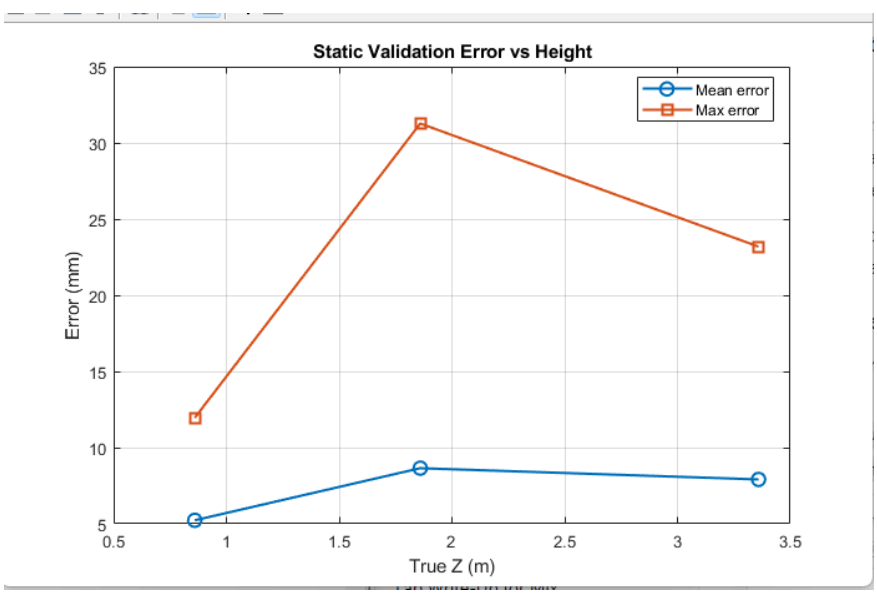


Fig. 8: Validation plot

2.4.7 Run Camera Accuracy

The Run Camera Accuracy option tests how camera placement error affects tracking accuracy. This is mainly a development and analysis tool.

When selected, the GUI introduces simulated camera position changes and reruns tracking tests to observe how measurement error changes. This can represent small camera setup differences, camera vibration, or environmental effects such as wind moving a camera slightly.

This option is useful because stereo triangulation depends heavily on accurate camera position and orientation. Even small changes in camera placement can affect the final measured X, Y, and Z position.

Important parameters:

-cameraPositionSigma
-cameraRotationSigma
-numSamples
-pair1_camBaseX
-pair1_camBaseY
-pair1_camBaseZ
-pair1_camPitchDeg
-pair2_camBaseX
-pair2_camBaseY
-pair2_camBaseZ
-pair2_camPitchDeg
-meanError
-RMSE
-maxError

Expected output:

-Camera accuracy table
-Mean position error
-RMSE
-Maximum error
-Error distribution plot
-Comparison of light, medium, and strong camera disturbance cases

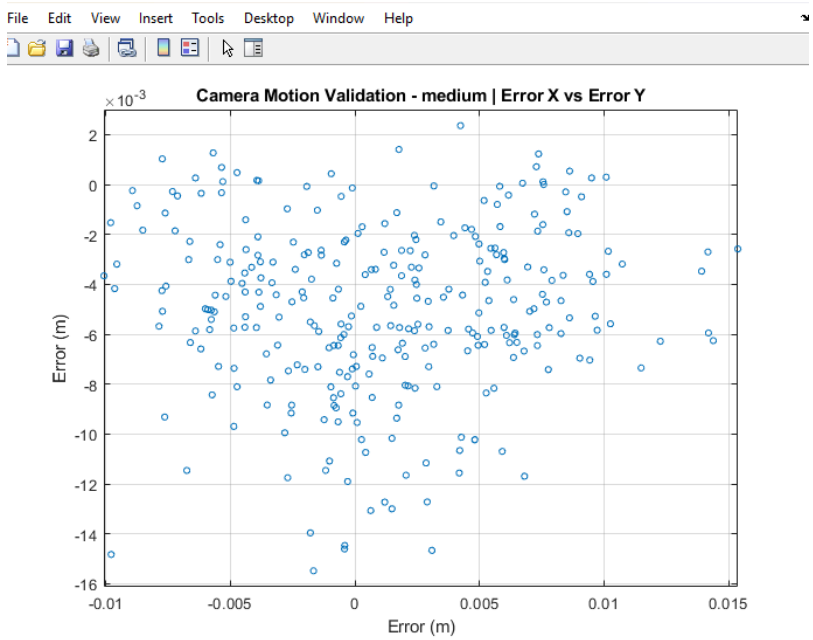


Fig. 9: Camera accuracy plot, ‘medium’ wind test

2.4.8 Clear

The Clear Trajectory option resets the GUI display. This may clear plots, remove previous trajectories, reset image axes, clear status text, and prepare the GUI for another test.

This option does not normally delete saved output files. It is mainly used to clean the visual display before running another shot or demonstration.

Expected output:

-Cleared court plot

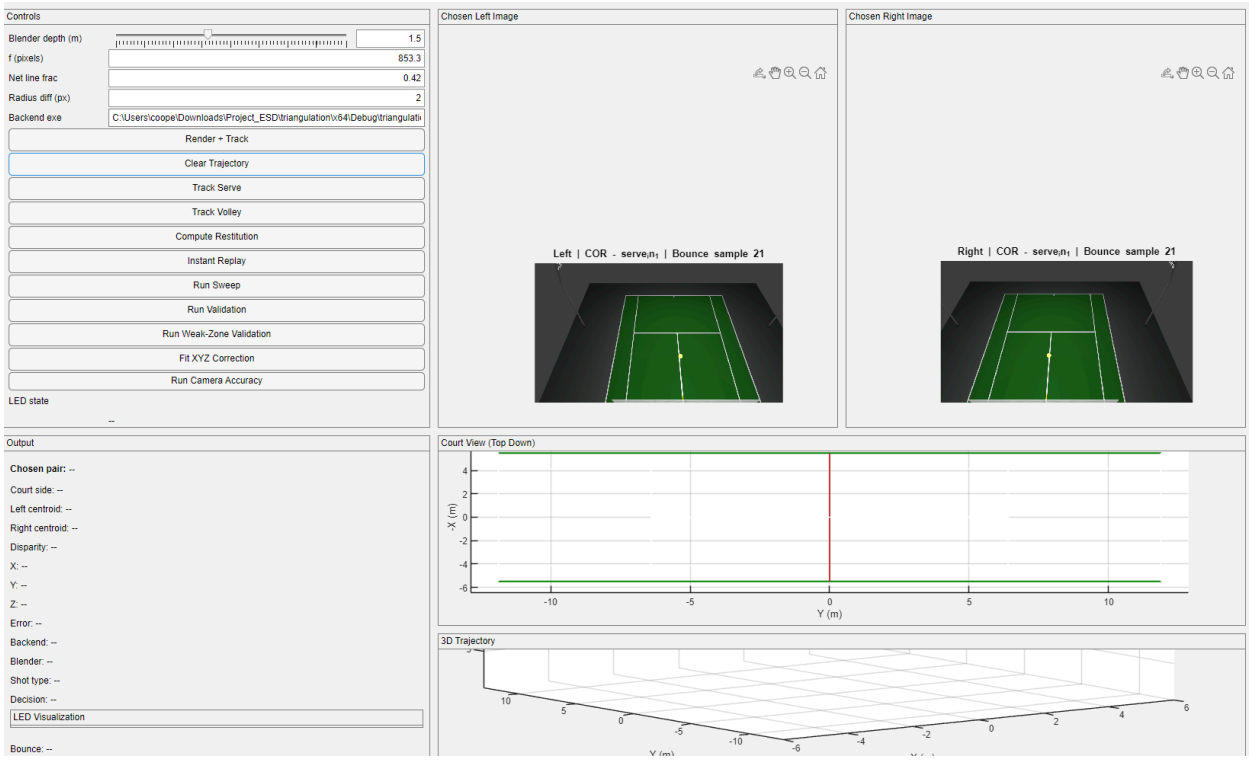


Fig. 10: GUI after pressing Clear Trajectory

3. System Setup

3.1 Required Software

The following software is required to run this project:

-MATLAB
-Blender 4.3.2
-Visual Studio 2022
-OpenCV
-C++ backend executable
-Project source files

MATLAB is used to run the GUI and control the system. Blender is used to render simulated court images. Visual Studio and OpenCV are used to compile and run the backend.

3.2 MATLAB Setup

Open MATLAB and navigate to the project folder. Add the project folder and subfolders to the MATLAB path. The main GUI can be started with:

GUI_Professional

The development GUI can be started with:

Project_GUI_Development_v1

3.3 Blender Setup

Open the Blender tennis court scene. Confirm that the court, ball, net, lights, and cameras are visible. Then run:

blenderServer.py

inside Blender's Python console or scripting workspace.

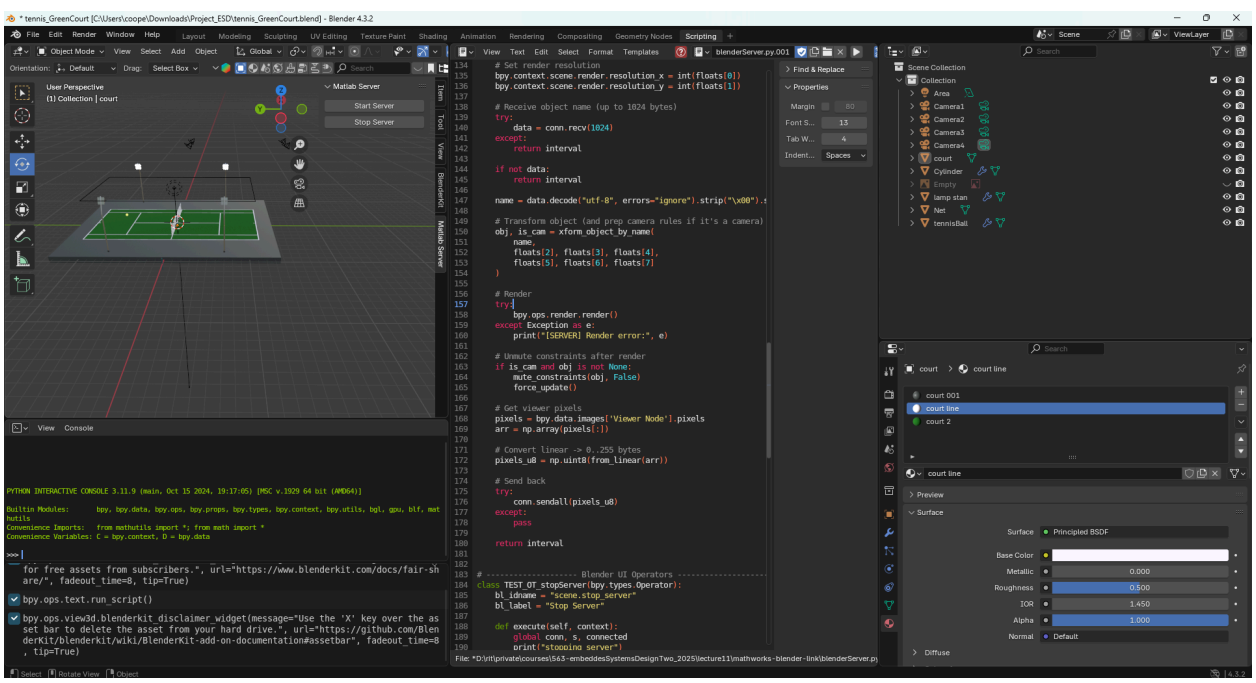


Fig. 11: Blender scene with court, cameras, python console, and 'MATLAB Server' opened

3.4 Backend Setup

The backend executable must exist before MATLAB can call it. A typical backend path is:

Project_ESD\triangulation\x64\Debug\triangulation.exe

This path **must** match the value used in the MATLAB GUI. If the executable is missing, rebuild the backend in Visual Studio.

3.5 Folder and Path Parameters

The following paths must be checked before running the system:

-projectRoot
-backendExe
-runRoot
-configPath
-resultPath
-courtOutputDir
-websiteImageDir

Example:

-projectRoot = 'C:\Users\coope\Downloads\Project_ESD';
-backendExe = fullfile(projectRoot, 'triangulation', 'x64', 'Debug', 'triangulation.exe');
-runRoot = fullfile(projectRoot, 'tracker_runs');
-courtOutputDir = fullfile(projectRoot, 'court_outputs');

These should be placed in a setup/configuration section of the manual, not buried inside the operation section.

4. Initial Use

4.1 Starting the Full System

This procedure is used to start the complete tracking system.

1. Open the Blender tennis court scene.
2. Run blenderServer.py. (confirm that the MATLAB server has started, refer to Fig. 11)
3. Open MATLAB.
4. Navigate to the Project_ESD folder.
5. Confirm that the backend executable path is correct.
6. Run GUI_Professional.
7. Press Render + Track to verify communication.

Expected result:

-Blender renders images.
-MATLAB displays the left and right camera views.
-The backend writes result.txt.
-The GUI displays measured XYZ values.

5. GUI Configuration

5.1 Professional GUI

The professional GUI is intended for final demonstration and field-style operation. It contains the core functions:

-Render + Track
-Track Serve
-Track Volley
-Instant Replay
-Court Plot
-Decision Display
-Saved Output Image
-Website Upload Helper

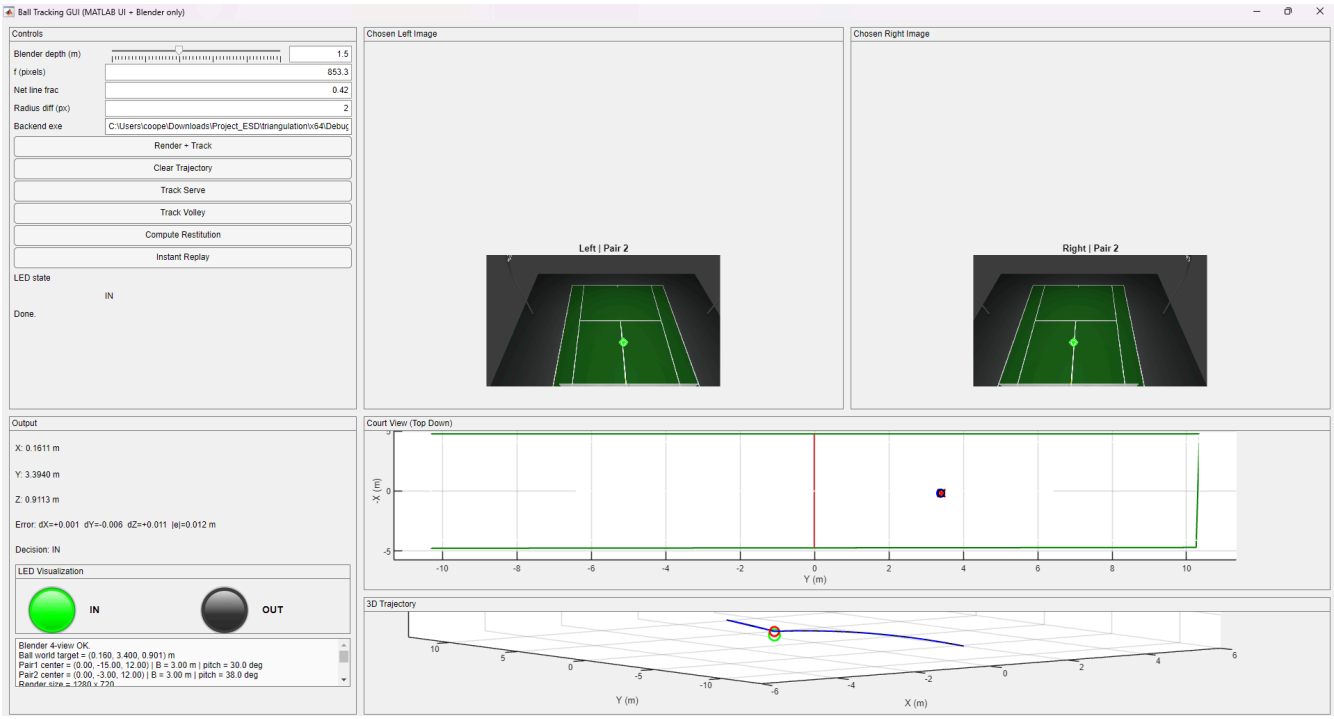


Fig. 12: Professional GUI

5.2 Development GUI

The development GUI contains additional diagnostic tools:

-Run Validation
-Run Camera Accuracy
-Static Ball Testing
-Court Color Testing
-Error vs Height Plots
-Heat Maps
-Failure Maps
-Shot Validation Tables

This version is used for debugging and system evaluation.

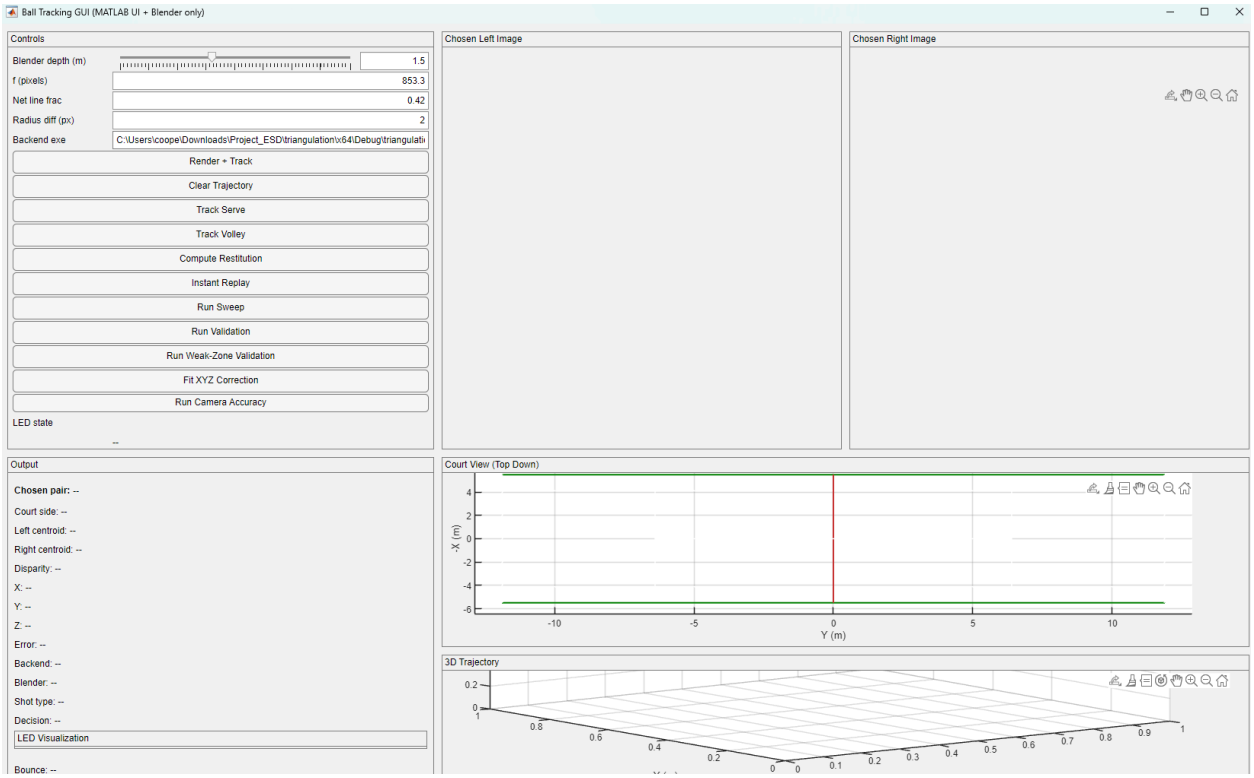


Fig. 13: Developmental GUI

5.3 Adjustable GUI Parameters

Important GUI parameters include:

-projectRoot
-backendExe
-ballXYZ
-groundZ
-courtGeom
-duration
-dt
-lineTolerance
-shotName
-shotNo
-lastShot

These **MUST** be checked before moving the project to another computer.

6. Blender Rendering Environment

6.1 Blender Scene Requirements

The Blender scene must include:

-Tennis court
-Court boundary lines
-Net
-Ball object
-Stereo cameras
-Lighting
-Ground plane

The ball object name **MUST** match the name expected by the Blender server and MATLAB code.

Common expected ball names:

-Ball
-tennisBall (currently the preset name)

6.2 Camera Setup

The system uses four cameras. In this four-camera setup, there are two stereo pairs:

Pair 1: Left camera and right camera
Pair 2: Left camera and right camera

Presetcamera values:

-pair1_camBaseX = 0
-pair1_camBaseY = -15
-pair1_camBaseZ = 12
-pair1_camPitchDeg = 30

-pair2_camBaseX = 0
-pair2_camBaseY = -3
-pair2_camBaseZ = 12
-pair2_camPitchDeg = 38

7. Backend and Triangulation

7.1 Backend Input Files

The backend normally receives:

-pair1_left.png
-pair1_right.png
-pair2_left.png
-pair2_right.png
-config.txt
-result.txt

Example backend command format:

-triangulation.exe pair1_left.png pair1_right.png pair2_left.png pair2_right.png config.txt
-result.txt

7.2 Camera and Intrinsic Parameters

Important intrinsic parameters:

-image_width = 1280
-image_height = 720
-fx_px = 853.3333
-fy_px = 853.3333
-cx = 639.5
-cy = 359.5

Important stereo parameters:

-pair1_baseline_m = 3
-pair2_baseline_m = 3

The values in MATLAB, Blender, and the backend configuration file **MUST** agree.

7.3 Detection Parameters

The backend uses HSV color detection (YCbCr), contour filtering, and circle checks (Hough circle transform).

Important detection parameters:

-HSV lower threshold
-HSV upper threshold
-minBlobArea
-maxBlobArea
-minimum circularity
-radius_diff_px
-expected ball radius range
-brightness threshold
-chroma tolerance

Common issue:

-no contours after yellow-ish HSV mask

This usually means the ball color, lighting, HSV threshold, or camera visibility needs adjustment.

7.4 Stereo Triangulation Parameters

The backend calculates disparity using:

-disparity = u_left - u_right

Depth is estimated with:

-Z = f * B / disparity

Important triangulation values:

-f = focal length in pixels
-B = stereo baseline
-cx = principal point X
-cy = principal point Y
-u_left = left image ball center X
-u_right = right image ball center X

Very small disparity values should be rejected because they can create unstable depth values.

7.5 Tracking Range Parameters

Tracking depends on the shot trajectory and camera visibility.

Important tracking parameters:

-duration
-dt
-initial position
-initial velocity
-gravity
-groundZ
-net height
-court limits
-camera field of view

A smaller dt gives more frames and smoother motion but increases runtime.

Example:

-duration = 2.0;
-dt = 0.05;
-t = 0:dt:duration;

8. Shot Classification

8.1 Court Geometry

Court geometry should be stored in:

-courtGeom

Important fields:

-courtGeom.polyX
-courtGeom.polyY

-courtGeom.netY
-courtGeom.groundZ
-courtGeom.lineWidth

8.2 Bounce Detection

The bounce point is usually stored as:

-shot.bounceXYZ

The bounce index may be stored as:

-shot.bounceIdx

The system may detect the bounce using the lowest Z point or a helper function such as:

-getPrimaryBounceIndex(...)

8.3 IN / OUT / NET Decision

A shot is **IN** if the bounce point is inside the court boundary or touching the court line.

A shot is **OUT** if the bounce point is outside the court boundary.

A shot is **NET** if the trajectory contacts the net or fails to clear the net region.

A small line tolerance should be used so that line-touching shots are correctly classified as **IN**:

-lineTolerance = 0.02;

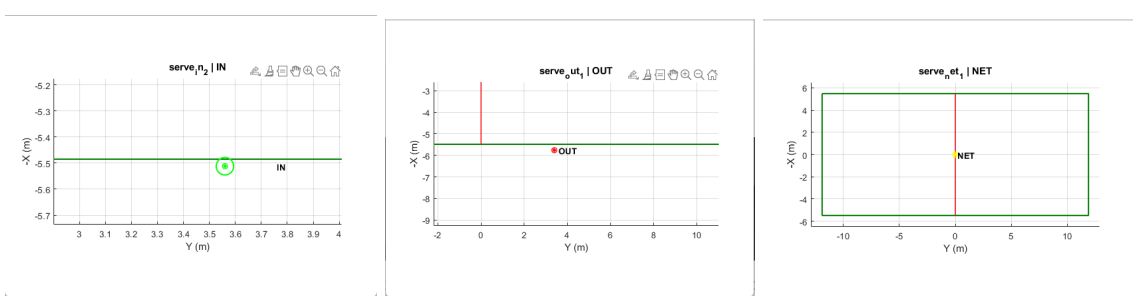


Fig. 14: Shot classification on 2-D tennis court

9. Website and SQL Output

9.1 lastShot Structure

The internal MATLAB result is stored in:

-lastShot

Important fields:

-lastShot.t
-lastShot.xyz
-lastShot.xyzMeas
-lastShot.xyzTrue
-lastShot.decision
-lastShot.bounceXYZ
-lastShot.selectedPair
-lastShot.shotName
-lastShot.shotNo

9.2 SQL Structure

The website-facing structure should be smaller and cleaner:

-sqlStruct.shot_type
-sqlStruct.shot_name
-sqlStruct.shot_no
-sqlStruct.decision
-sqlStruct.bounce_x
-sqlStruct.bounce_y
-sqlStruct.bounce_z
-sqlStruct.selected_pair
-sqlStruct.court_image

For the image path:

```
[~, imgName, imgExt] = fileparts(image_source);
sqlStruct.court_image = ['../images/' imgName imgExt];
```

The stored SQL image path **MUST** match the folder used by the website frontend.

10. User Sided GUI

10.1 Basic Requirements

A machine will be needed to host the SQL server and to run the python scripts for the User GUI's backend and frontend. On installation of the system, the following will need to be done one time at first boot:

1. Navigate to the directory webdev/flask_tracker
1. Create a virtual environment using Python 3.12 or higher. Any previous version will not work. Create a virtual environment using the command `python -m venv .venv` on the machine's Linux command line.
2. Activate the virtual environment using the command `source .venv/bin/activate`.
3. Once the virtual environment is activated, run the install.sh script inside of webdev/flask_tracker/. This will download the needed Python libraries for the User GUI.

10.2 My SQL Database

When MATLAB pushes the SQL structure to MySQL, the database will convert this structure into a table of data that can be retrieved by the backend. The backend python script will handle converting the SQL tables and providing them to the frontend so the users can access them. This connection will begin automatically when the GUI application is started.

10.3 Client Facing Site

The application will be accessible through a webserver in the future once fully implemented. To start the application / website for the User GUI, run the startup.sh script in the webdev/flask_tracker folder. This will begin running the application for the User GUI in an application window. It will automatically update a list with entries from the SQL, and entries will be clickable and will populate the screen with the information provided in the SQL shot structure seen in 9.2.

Once implemented, this will also begin a webserver that will be accessible to anyone with a link to view the data from the serve / volley sequence from their own device in a much simpler format than the MATLAB GUI for accessibility purposes to general audiences watching the game and want more information about a particular ruling.

Fig. 15: Client facing User GUI

11. Troubleshooting

Problem	*Likely Cause*	*Fix*
MATLAB cannot connect to Blender	Blender server not running	Run blenderServer.py
Backend results missing	Wrong executable path	Check backendExe
No contours found	HSV threshold mismatch	Adjust HAV range/color detection
Pair 1 fails, Pair 2 works	Ball is outside Pair 1 view	Use pair selected output
Website image missing	Wrong relative path	Check ../images/ path
Website / application not accessable	Python scripts not started	Run startup.sh

Glossary

-Backend:
The C++/OpenCV executable that detects the ball and triangulates position.

-Blender Server:
The Python script running inside Blender that receives MATLAB commands.

-Disparity:
The difference between the ball's horizontal pixel location in the left and right camera images.

-GroundZ:
The court surface height used for bounce detection.

-lastShot:
The MATLAB structure storing the most recent shot result.

-Selected Pair:
The stereo camera pair used by the backend for the successful measurement.

-Triangulation:
The process of calculating a 3D point from two camera images.

Appendices

Appendix A.1 Recommended Demo Procedure

1. Open Blender.
2. Run blenderServer.py.
3. Open MATLAB.
4. Navigate to the Project folder.
5. Run GUI_Professional.
6. Press Render + Track.
7. Press Track Serve.
8. Press Track Volley.
9. Press Instant Replay.
10. Show saved court PNG.
11. Show SQL/website output if needed.

Appendix A.2 Important Parameters Checklist

-projectRoot
-backendExe
-runRoot
-configPath
-resultPath
-courtOutputDir
-image_width
-image_height
-fx_px
-fy_px
-cx
-cy
-baseline
-camera position
-camera pitch
-groundZ
-courtGeom.netY
-lineTolerance
-HSV thresholds
-minBlobArea
-radius_diff_px